ProLoSA: Improving Compressed LoRA with Sparse Residual Adaptation

Anonymous ACL submission

Abstract

Low-rank adaptation (LoRA) is a widely used parameter-efficient fine-tuning method, yet its update space still contains substantial redundancy. Recent compressed LoRA methods improve efficiency by reconstructing LoRA parameters from a low-dimensional latent space, but aggressive compression can over-constrain adaptation and miss locally important directions. We propose ProLoSA, a projected LoRA framework with a sparse adapter. ProLoSA represents the full LoRA parameter vector as the sum of a globally projected latent parameter and a routed sparse residual, preserving the regularization benefit of compression while restoring local flexibility. We further provide a bias–variance analysis showing why compressed LoRA can outperform classical LoRA when the downstream update is well aligned with the compressed subspace, and how sparse residual correction reduces projection bias. Experiments on synthetic validation, natural language understanding, mathematical reasoning, and instruction tuning show that ProLoSA achieves a strong performance–parameter tradeoff compared with LoRA, Uni-LoRA, and other PEFT baselines.

1 Introduction

Large language models (LLMs) have achieved remarkable success across a wide range of natural language understanding and generation tasks. However, adapting these models to downstream tasks via full fine-tuning remains computationally and storage intensive, especially when many task-specific, domain-specific, or user-specific variants must be maintained. Parameter-efficient fine-tuning (PEFT) addresses this challenge by updating only a small subset of parameters while keeping the pretrained backbone frozen. Among existing PEFT methods, Low-Rank Adaptation (LoRA) has emerged as one of the most influential approaches due to its simplicity, strong empirical performance, and zero additional inference latency after weight merging (Hu et al., 2022).

Despite its effectiveness, standard LoRA is still not fully efficient in its parameterization of task-specific updates. A growing body of work suggests that the effective degrees of freedom required for adapting pretrained models often lie in a much lower-dimensional space than the ambient parameter space, indicating substantial redundancy even within fine-tuning updates (Aghajanyan et al., 2021). At the same time, empirical studies have shown that different layers and modules contribute unequally to downstream adaptation, implying that allocating an identical budget everywhere can be suboptimal (Zhang et al., 2023). Moreover, although LoRA is highly parameter-efficient, there often remains a noticeable gap between LoRA-style adaptation and full fine-tuning, which suggests that improving the expressiveness and optimization properties of low-rank updates remains an important direction (Liu et al., 2024).

These observations have motivated a line of research beyond vanilla LoRA, shifting attention from merely reducing rank to redesigning how the adaptation space itself is parameterized, shared, and optimized. A particularly appealing perspective is to view LoRA updates as arising from a compressed latent parameterization, from which the full update is reconstructed through a structured mapping. Under this view, the key question is not only *how many* trainable parameters are used, but also *how* these parameters are organized and projected back to the original LoRA space. This perspective naturally highlights the promise of global parameter sharing and structured reconstruction for achieving extreme parameter efficiency.

However, existing compressed or shared LoRA parameterizations still face a fundamental tension between efficiency and flexibility. Aggressive sharing can greatly reduce the number of trainable parameters, but it may also over-constrain the up-

date space and ignore the heterogeneous adaptation needs of different layers, modules, or parameter groups. Conversely, methods with stronger local flexibility often reintroduce substantial overhead or fail to fully exploit global redundancy. This raises a central question: *can we preserve the compression benefits of globally parameterized LoRA while restoring sufficient flexibility to better match the non-uniform structure of downstream adaptation?*

In this paper, we answer this question affirmatively and propose PROLOSA (Projected LoRA with Sparse Adapter), a new compressed adaptation framework built on top of Uni-LoRA. ProLoSA retains the core idea of global low-dimensional adaptation, while improving the reconstruction of the full LoRA parameter space through two complementary designs. First, we augment the compressed parameterization with a sparse adapter, which provides a lightweight residual pathway to capture important coordinates that are difficult to express through purely shared compression. Second, we introduce a dynamic bucket reassignment mechanism, which allows parameters to be rerouted across buckets during training, thereby alleviating the rigidity of static parameter sharing and enabling the compressed representation to better match the evolving optimization landscape. Together, these designs yield a more expressive yet still highly efficient adaptation space.

Beyond the method itself, we also provide a theoretical perspective on why compressed LoRA parameterizations can, in some regimes, outperform classical LoRA rather than merely approximating it. Our analysis shows that a properly designed compressed parameterization can introduce a favorable inductive bias by restricting adaptation to a better-structured subspace, reducing redundant degrees of freedom and improving the variance–bias trade-off of the learned update. This provides a principled explanation for the empirical advantage of compressed LoRA variants and helps clarify why stronger compression does not necessarily imply weaker adaptation.

Extensive experiments show that PROLOSA remains competitive with strong PEFT baselines under comparable or even smaller trainable parameter budgets. These results suggest that the next step beyond extreme compression is not merely to use fewer parameters, but to design better structured reconstruction mechanisms and better adaptive allocation strategies for low-rank adaptation.

Our contributions are summarized as follows:

- We propose PROLOSA, a new LoRA-based adaptation framework that combines a routed sparse adapter with dynamic bucket reassignment, achieving a better balance between global compression and local flexibility.
- We provide a theoretical explanation for why compressed LoRA parameterizations can outperform classical LoRA, showing that structured compression can act as an effective inductive bias and yield a better bias–variance trade-off during adaptation.
- We empirically demonstrate that PROLOSA achieves a stronger performance–parameter trade-off than Uni-LoRA and competitive PEFT baselines across multiple downstream tasks.

2 Related Work

Parameter-efficient fine-tuning and low-rank adaptation. Parameter-efficient fine-tuning (PEFT) aims to adapt pretrained language models by updating only a small subset of parameters while keeping the backbone frozen. Early PEFT methods mainly relied on task-specific modules such as adapters or lightweight parameter subsets, substantially reducing the cost of downstream adaptation. Among these approaches, LoRA (Hu et al., 2022) has become one of the most influential methods by representing weight updates as low-rank matrices, achieving strong performance with no additional inference latency after merging. Building on LoRA, subsequent studies have explored how to improve the effectiveness of low-rank adaptation under limited budgets. AdaLoRA (Zhang et al., 2023) observes that different weight matrices have unequal importance and proposes adaptive budget allocation across layers, while DoRA (Liu et al., 2024) revisits the gap between LoRA and full fine-tuning through weight decomposition and improves the expressiveness of low-rank updates.

Compressed and shared parameterizations of LoRA. Beyond improving vanilla low-rank adaptation, another line of work focuses on further compressing the trainable parameter space of LoRA. Tied-LoRA (Renduchintala et al., 2024) introduces cross-layer weight tying and selective training to reduce trainable parameters. VeRA (Kopiczko et al., 2024) replaces trainable low-rank matrices with frozen shared random matrices and learns only

lightweight scaling vectors. VB-LoRA (Li et al., 2024) pushes parameter sharing further through a global vector bank, while LoRA-XS (Bałazy et al., 2025) inserts an extremely small trainable matrix between frozen low-rank factors derived from pre-trained weights. More recently, Uni-LoRA (Li et al., 2025) unifies many of these methods under a global subspace projection view, in which LoRA parameters are reconstructed from a shared low-dimensional vector through a projection matrix. These methods demonstrate that substantial redundancy still exists even within the LoRA parameter space. However, their emphasis is primarily on continuous low-dimensional compression and global sharing, rather than modeling sparse, localized residual updates that may remain important for downstream adaptation.

Sparse adaptation and hybrid sparse–low-rank methods. A complementary research direction studies sparsity as the main inductive bias for efficient adaptation. SparseAdapter (He et al., 2022) improves adapter tuning by pruning adapter parameters and shows that sparse adapters can outperform dense adapters under the same budget. In the context of large language models, sparse fine-tuning methods such as SIFT (Song et al., 2024) and SpIEL (Ansell et al., 2024) directly optimize a sparse subset of parameter updates, showing that sparsity alone can be a competitive PEFT strategy. Hybrid approaches combine the strengths of low-rank and sparse adaptation. Most notably, RoSA (Nikdan et al., 2024) jointly trains low-rank and highly sparse components on top of frozen pre-trained weights, motivated by robust principal component analysis. Our work is most closely related to this hybrid line, but differs in a key aspect: instead of adding sparse corrections to a standard low-rank or full-parameter update space, we start from the Uni-LoRA view of globally shared low-dimensional reconstruction and introduce a sparse adapter branch as a corrective mechanism. In this sense, our method combines the global compression advantages of Uni-LoRA with the flexibility of sparse residual adaptation.

3 Proposed Methodology

In this section, we first present the theoretical motivation behind PROLOSA. Our goal is to explain why a compressed LoRA parameterization can in principle outperform classical LoRA, rather than merely serving as a more parameter-efficient approximation. We then introduce the full design of PROLOSA, including the routed sparse adapter and dynamic bucket reassignment mechanism.

3.1 Theoretical Motivation

We start from a unified view of LoRA-style adaptation in the full parameter space. Let $\theta_D \in \mathbb{R}^D$ denote the vectorized LoRA parameters. Classical LoRA directly optimizes θ_D, whereas compressed LoRA methods instead optimize a lower-dimensional latent parameter $\theta_d \in \mathbb{R}^d$ with $d \ll D$, and reconstruct the full update through a structured projection:

$$\theta_D = P\theta_d, \tag{1}$$

where $P \in \mathbb{R}^{D\times d}$ is a fixed projection matrix. Under this view, classical LoRA can be regarded as the special case with $P = I$. Therefore, compressed LoRA is not merely "using fewer parameters", but optimizing within a restricted subspace $\mathcal{S} = \mathrm{col}(P)$ of the original LoRA parameter space.

This perspective raises a natural question: why can such a restriction improve generalization rather than simply hurt expressiveness? To answer this question, we adopt a local quadratic approximation of the population risk around the LoRA optimum. Let $\theta^\star$ denote the optimal solution in the full LoRA space. Around $\theta^\star$, the risk can be approximated as

$$R(\theta) \approx R(\theta^\star) + \frac{1}{2}(\theta-\theta^\star)^\top H(\theta-\theta^\star), \qquad H \succeq 0, \tag{2}$$

where H is the local curvature matrix. Suppose the empirical solution of classical LoRA is

$$\hat{\theta}_L = \theta^\star + \xi, \qquad \mathbb{E}[\xi] = 0, \qquad \mathrm{Cov}(\xi) = \Sigma. \tag{3}$$

Then its expected excess risk is approximately

$$\mathbb{E}[R(\hat{\theta}_L)] - R(\theta^\star) \approx \frac{1}{2}\mathrm{tr}(H\Sigma). \tag{4}$$

For a compressed parameterization, optimization is restricted to the subspace $\mathcal{S} = \mathrm{col}(P)$. The corresponding population optimum becomes the H-weighted projection of $\theta^\star$ onto $\mathcal{S}$:

$$\theta^\star_{\mathcal{S}} = \arg\min_{\theta\in\mathcal{S}} (\theta - \theta^\star)^\top H(\theta - \theta^\star). \tag{5}$$

Assume the empirical solution in this subspace is

$$\hat{\theta}_U = \theta^\star_{\mathcal{S}} + P\zeta, \qquad \mathbb{E}[\zeta] = 0, \qquad \mathrm{Cov}(\zeta) = \Sigma_z. \tag{6}$$

Its expected excess risk can then be decomposed as

$$\mathbb{E}[R(\hat{\theta}_U)] - R(\theta^\star) \approx \underbrace{\frac{1}{2}(\theta^\star_{\mathcal{S}} - \theta^\star)^\top H(\theta^\star_{\mathcal{S}} - \theta^\star)}_{\text{bias}} + \underbrace{\frac{1}{2}\mathrm{tr}(HP\Sigma_z P^\top)}_{\text{variance}}, \tag{7}$$

This decomposition immediately shows that compressed LoRA can outperform classical LoRA whenever the bias introduced by the subspace constraint is smaller than the variance reduction it brings, namely when

$$(\theta^\star_{\mathcal{S}} - \theta^\star)^\top H(\theta^\star_{\mathcal{S}} - \theta^\star) + \mathrm{tr}(HP\Sigma_z P^\top) < \mathrm{tr}(H\Sigma). \tag{8}$$

In other words, a compressed parameterization may improve generalization not because it is more expressive than LoRA, but because it excludes redundant, noisy, or task-irrelevant directions and thereby induces a more favorable bias–variance trade-off.

This interpretation becomes particularly natural in the context of fine-tuning pretrained models. Let the downstream optimum be written as

$$W^\star = W_0 + \Delta^\star, \tag{9}$$

where W_0 is the frozen pretrained backbone and $\Delta^\star$ is the task-specific residual update to be learned. LoRA-style methods do not need to represent the full model parameters from scratch; they only need to model the residual adaptation $\Delta^\star$. If the subspace constraint is imposed on this residual update, then the bias term can be bounded as

$$\mathsf{Bias}_{\mathcal{S}} = \frac{1}{2}\|\Delta^\star - \Delta^\star_{\mathcal{S}}\|^2_{H_\Delta} \le \frac{\lambda_{\max}(H_\Delta)}{2}\varepsilon^2\|\Delta^\star\|_2^2, \tag{10}$$

where ε denotes the projection error ratio. Since in practice $\|\Delta^\star\| \ll \|W^\star\|$ for pretrained models, constraining the residual update is much less harmful than directly constraining the full parameter vector. This explains why compressed LoRA methods can benefit from subspace regularization while avoiding the severe degradation often observed in direct compression schemes over full model weights.

The above analysis also clarifies why the structure of the projection matrix P is crucial. A good compressed parameterization should not merely reduce dimensionality, but should do so in a way that preserves the useful geometry of LoRA updates. This is precisely why Uni-LoRA emphasizes three desirable properties of P: *globality*, *uniformity*, and *isometry*. Globality enables the same latent coordinates to support correlated adaptations across layers and modules, which can reduce projection bias when the true update exhibits cross-layer shared structure. Uniformity prevents a small number of latent buckets from being overloaded, thereby avoiding highly uneven approximation error across the full parameter space. Isometry, typically enforced through $P^\top P = I$, preserves distances within the latent subspace and prevents additional geometric distortion introduced by the projection itself. Together, these properties make compressed adaptation more likely to retain the shared signal while discarding harmful degrees of freedom.

The theory above provides the key motivation for PROLOSA. If a well-structured compressed subspace can outperform classical LoRA by improving the bias–variance trade-off, then the next question is how to further enhance the flexibility of such a compressed parameterization without losing its regularization benefit. Our method addresses this by combining global compressed adaptation with a sparse residual pathway and a dynamic reassignment mechanism, so that the model can preserve the advantages of structured compression while adapting more effectively to heterogeneous parameter importance during training.

3.2 PROLOSA Overview

We now introduce the full design of PROLOSA, short for *Projected LoRA with Sparse Adapter*. The method consists of two complementary branches: a compressed LoRA branch that captures the dominant shared adaptation structure through a low-dimensional latent parameterization, and a sparse adapter branch that provides a lightweight correction in the original LoRA parameter space.

Consider a pretrained model with frozen parameters W_0, and let $\mathcal{L}$ denote the set of linear layers to be adapted. For each target layer $\ell \in \mathcal{L}$, standard LoRA introduces a low-rank update

$$\Delta W_\ell = B_\ell A_\ell, \tag{11}$$

where $A_\ell \in \mathbb{R}^{r_\ell \times d_\ell^{\text{in}}}$ and $B_\ell \in \mathbb{R}^{d_\ell^{\text{out}} \times r_\ell}$ are trainable factors. Following the compressed LoRA view in Section 3.1, we vectorize all LoRA parameters across all adapted layers into a single full-space parameter

$$\theta_D = \mathrm{vec}\big(\{A_\ell, B_\ell\}_{\ell \in \mathcal{L}}\big) \in \mathbb{R}^D. \tag{12}$$

Instead of directly optimizing θ_D, PROLOSA parameterizes it as

$$\theta_D = P\theta_d + \theta_s, \tag{13}$$

where $\theta_d \in \mathbb{R}^d$ with $d \ll D$ is the compressed latent parameter, $P \in \mathbb{R}^{D\times d}$ is the fixed projection matrix inherited from the compressed LoRA design, and $\theta_s \in \mathbb{R}^D$ is a sparse residual term.

The role of the two branches is conceptually different. The projected term $P\theta_d$ captures the globally shared and low-dimensional part of the adaptation, which benefits from the regularization effect analyzed in Section 3.1. The sparse term θ_s is used to compensate for local approximation errors, especially for a small number of coordinates that are important for the downstream task but poorly represented by the projected subspace. As a result, PROLOSA extends the pure compressed subspace $\mathrm{col}(P)$ to a richer structured space:

$$\mathcal{S}_{\text{PROLOSA}} = \{P\theta_d + \theta_s \mid \theta_d \in \mathbb{R}^d,\ \theta_s \text{ is sparse}\}. \tag{14}$$

This parameterization preserves the efficiency of compressed LoRA while relaxing its rigidity through a highly parameter-efficient residual pathway.

After constructing θ_D, we reshape its corresponding slices back to the LoRA factors of each layer. For notational convenience, let $\mathcal{U}_\ell(\cdot)$ denote the unpacking operator that maps the relevant slice of θ_D back to the pair (A_ℓ, B_ℓ). Then the final adaptation applied to layer ℓ is obtained from

$$(A_\ell, B_\ell) = \mathcal{U}_\ell(\theta_D), \qquad \Delta W_\ell = B_\ell A_\ell. \tag{15}$$

The resulting model is evaluated using $W_\ell = W_{\ell,0} + \Delta W_\ell$ in the same way as standard LoRA, and the learned update can be merged into the backbone weights at inference time.

3.3 Routed Sparse Adapter

A central component of PROLOSA is the sparse residual term in Eq. (13). A naive implementation would optimize a dense residual vector in $\mathbb{R}^D$, which would largely defeat the purpose of parameter efficiency. Instead, we parameterize the residual through a routed sparse adapter.

Specifically, let $K \ll D$ denote the sparse budget, i.e., the number of nonzero residual coordinates allowed in the full LoRA space. We introduce a routing matrix

$$R \in \{0,1\}^{D\times K}, \tag{16}$$

whose j-th column contains exactly one nonzero entry. Equivalently, each sparse adapter parameter is assigned to one coordinate in the full LoRA parameter space through an index mapping

$$\pi : \{1,\dots,K\} \to \{1,\dots,D\}. \tag{17}$$

Let $\alpha \in \mathbb{R}^K$ denote the trainable sparse coefficients. The sparse residual is then written as

$$\theta_s = R\alpha = \sum_{j=1}^{K} \alpha_j e_{\pi(j)}, \tag{18}$$

where $e_{\pi(j)}$ is the standard basis vector corresponding to the routed coordinate. By construction, θ_s has at most K nonzero entries, and the total ProLoSA parameterization becomes

$$\theta_D = P\theta_d + R\alpha. \tag{19}$$

This formulation makes the role of the sparse adapter explicit. The compressed branch $P\theta_d$ provides a low-dimensional global reconstruction of the LoRA update, while the routed sparse branch $R\alpha$ injects a small number of free coordinates directly into the original parameter space. Therefore, the sparse adapter acts as an error-correction mechanism for the projected approximation. From the perspective of the bias–variance analysis in Section 3.1, the projected branch reduces estimation variance by restricting the solution to a structured subspace, whereas the sparse branch reduces the projection bias by selectively recovering important coordinates outside or poorly aligned with that subspace.

Another advantage of Eq. (19) is that it decouples *where* the residual is placed from *how much* correction is needed. The routing matrix R specifies the support locations of the sparse correction, while α controls the correction magnitude. This makes the sparse branch substantially more parameter-efficient than adding an unrestricted dense residual. In particular, the number of trainable parameters introduced by the sparse branch is only K, and the overall trainable parameter count of PROLOSA becomes

$$d + K, \tag{20}$$

which remains far smaller than the full LoRA parameter count D when both d and K are chosen conservatively.

In practice, the routed sparse adapter can be viewed as a lightweight complement to compressed

LoRA. When the projected subspace already captures the dominant shared structure of the downstream update, only a small number of residual coordinates are needed to compensate for the remaining mismatch. This design is therefore well aligned with the empirical observation that adaptation updates are globally redundant but locally heterogeneous: most coordinates can be compressed and shared, while a small fraction benefit from explicit correction.

3.4 Training Objective and Optimization

Given the parameterization in Eq. (19), PROLOSA is trained by optimizing the compressed latent parameter θ_d and the sparse coefficients α, while keeping the pretrained backbone W_0 frozen. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the downstream training set, and let $f(x; W_0, \theta_d, \alpha)$ denote the model output under the ProLoSA update. The training objective is

$$\min_{\theta_d, \alpha} \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(x_i; W_0, \theta_d, \alpha), y_i\big), \qquad (21)$$

where $\ell(\cdot, \cdot)$ is the task loss.

During optimization, the projection matrix P is fixed, so that the compressed branch retains the geometric properties discussed in Section 3.1, such as globality, uniformity, and isometry when applicable. The routing matrix R is also fixed once the sparse support is determined, and only the coefficient vector α is trained. This keeps the optimization simple and stable: the model learns a global compressed update through θ_d and a sparse residual correction through α, without introducing an additional dense branch or modifying the frozen backbone.

The forward and backward passes follow the same high-level workflow as standard LoRA. At each training step, we first reconstruct the full LoRA parameter vector using Eq. (19), then unpack it into layer-wise factors (A_ℓ, B_ℓ), and finally compute the LoRA updates $\Delta W_\ell = B_\ell A_\ell$ for all adapted layers. Since the sparse branch is defined directly in the LoRA parameter space rather than in the backbone parameter space, PROLOSA preserves the modularity and mergeability of LoRA-style adaptation. After training, the reconstructed LoRA update can be merged into the base model weights for deployment, introducing no additional inference latency beyond standard merged LoRA.

Overall, PROLOSA can be interpreted as a structured compromise between pure compressed adaptation and unrestricted LoRA. The compressed branch provides a strong low-dimensional inductive bias, while the routed sparse adapter supplies a small amount of targeted flexibility. In Section 4.2, we design a synthetic experiment that is mathematically aligned with this analysis and directly verify the predicted bias–variance behavior of LoRA and Uni-LoRA under controlled subspace mismatch and sample size.

4 Experiments

We evaluate PROLOSA on four complementary settings: a synthetic theory-validation setup, natural language understanding, mathematical reasoning, and instruction tuning. The synthetic setup is designed to directly test the bias–variance predictions derived in Section 3.1 under a fully controllable setting, while the real downstream benchmarks assess whether the same advantages transfer to practical fine-tuning tasks. Together, these experiments allow us to evaluate whether the proposed hybrid parameterization is effective not only in a theory-aligned quadratic regime, but also on challenging real-world adaptation problems under tight parameter budgets.

4.1 Experimental Settings

Benchmarks. We consider three groups of benchmarks. For natural language understanding, we evaluate on the GLUE benchmark (Wang et al., 2018), including SST-2, MRPC, CoLA, QNLI, RTE, and STS-B. For mathematical reasoning, we evaluate on GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). For open-ended generation, we perform instruction tuning on Cleaned Alpaca (Taori et al., 2023; Yahma, 2023) and evaluate the resulting models on MT-Bench (Zheng et al., 2023). Optionally, we additionally report AlpacaEval 2 (Dubois et al., 2024) to provide a more robust automatic assessment of instruction-following quality.

Backbone models. Following prior PEFT studies, we use RoBERTa-base and RoBERTa-large (Liu et al., 2019) for GLUE, and adopt decoder-only LLMs for generative tasks. For mathematical reasoning, we use [Mistral-7B (Jiang et al., 2023) / Gemma-7B (Gemma Team, 2024) / your final choice]. For instruction tuning, we use [Llama-2-7B / Llama-2-13B (Touvron et al., 2023) / your

final choice]. For the LLM experiments, we follow the QLoRA training recipe (Dettmers et al., 2023) unless otherwise stated, so that all methods are compared under the same low-memory setting.

Baselines. We compare PROLOSA against the following baselines: (1) **LoRA** (Hu et al., 2022), the standard low-rank adaptation method; (2) **AdaLoRA** (Zhang et al., 2023), which dynamically reallocates rank across layers; (3) **DoRA** (Liu et al., 2024), which improves LoRA through weight decomposition; (4) **VeRA** (Kopiczko et al., 2024), which shares frozen random low-rank matrices and trains scaling vectors; (5) **FourierFT** (Gao et al., 2024), which parameterizes updates in the Fourier domain; (6) **Uni-LoRA** (Li et al., 2025), which reconstructs the full LoRA parameter vector from a shared low-dimensional latent vector; and (7) **RoSA** (Nikdan et al., 2024), a hybrid low-rank+sparse adaptation baseline. For fairness, all methods are applied to the same target modules in each backbone.

Implementation details. For standard LoRA-style baselines, we use rank $r \in \{[\,]\}$ and tune it following the recommended settings of each method. For PROLOSA, the total number of trainable parameters is

$$B = d + K,$$

where d is the dimension of the compressed latent parameter and K is the sparse budget. Unless otherwise stated, we tune d and K under a fixed total budget B on the validation set, and report the best configuration for each task. We initialize the compressed branch and sparse branch following the procedure described in Section 3. To avoid confounding factors, we keep the optimizer, scheduler, training epochs, and batch size aligned across methods whenever possible.

Evaluation metrics. For GLUE, we report accuracy on SST-2, MRPC, QNLI, and RTE, Matthew's correlation on CoLA, and Pearson correlation on STS-B. For GSM8K and MATH, we report exact-match accuracy. For instruction tuning, we report the average MT-Bench score (Zheng et al., 2023). If AlpacaEval 2 is included, we report the length-controlled win rate (Dubois et al., 2024). We also report trainable parameter count, peak GPU memory, and training time to highlight the efficiency–performance trade-off.

4.2 Synthetic Validation of the Theory

We first design a synthetic experiment to directly verify the theoretical analysis in Section 3.1. The goal of this experiment is not to maximize downstream performance, but to test whether the empirical behavior of LoRA and Uni-LoRA follows the bias–variance decomposition predicted by the quadratic risk model. In particular, we aim to verify three predictions: (1) Uni-LoRA can outperform LoRA when the true update is well aligned with the compressed subspace; (2) the advantage of Uni-LoRA shrinks as the subspace mismatch increases; and (3) the relative behavior of the two methods is governed by the same bias–variance trade-off predicted by our theory.

Synthetic setup. We consider a teacher–student linear regression problem in a D-dimensional parameter space. Let $x \sim \mathcal{N}(0, I_D)$ and

$$y = x^\top \theta^\star + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_y^2), \tag{22}$$

where $\theta^\star \in \mathbb{R}^D$ is the ground-truth task-specific update. Under this isotropic Gaussian design, the population squared loss is

$$R(\theta) = \frac{1}{2}\mathbb{E}_{x,y}\left[(x^\top\theta - y)^2\right] = R(\theta^\star) + \frac{1}{2}\|\theta - \theta^\star\|_2^2, \tag{23}$$

which is exactly the quadratic risk model analyzed in Section 3.1 with $H = I$.

To instantiate the Uni-LoRA subspace, we sample a random projection matrix

$$P \in \mathbb{R}^{D\times d}, \qquad P^\top P = I_d, \tag{24}$$

by drawing a Gaussian matrix and orthonormalizing its columns. We then construct the ground-truth update as

$$\theta^\star = P z^\star + \alpha s_\perp, \tag{25}$$

where $z^\star \sim \mathcal{N}(0, I_d)$, $s_\perp \in \mathbb{R}^D$ is a unit vector orthogonal to $\mathrm{col}(P)$, and $\alpha \geq 0$ controls the mismatch between the true update and the Uni-LoRA subspace. When $\alpha = 0$, the true update lies exactly in the compressed subspace; as α increases, the projection bias of Uni-LoRA increases accordingly.

Compared methods. We compare the following two methods in this synthetic setting:

- **LoRA-space estimator**: directly fits an unconstrained parameter $\theta \in \mathbb{R}^D$ by minimizing the empirical squared loss.

- **Uni-LoRA-space estimator**: restricts the parameterization to $\theta = Pz$ with $z \in \mathbb{R}^d$ and fits z by minimizing the same empirical loss.

For both methods, we use the same training set size n and evaluate the population excess risk on a large independently sampled test set.

Theoretical prediction. Under the quadratic model in Section 3.1, LoRA has negligible approximation bias but higher estimation variance, whereas Uni-LoRA exhibits the decomposition

$$\mathbb{E}[R(\hat{\theta}_U)] - R(\theta^\star) \approx \frac{1}{2}\|\theta^\star - PP^\top\theta^\star\|_2^2 + \frac{1}{2}\mathrm{tr}(P\Sigma_z P^\top). \tag{26}$$

For the construction in Eq. (25), the bias term is exactly

$$\frac{1}{2}\|\theta^\star - PP^\top\theta^\star\|_2^2 = \frac{1}{2}\alpha^2. \tag{27}$$

Thus, the theory predicts that Uni-LoRA should outperform LoRA when α is small enough that the reduction in variance dominates the additional projection bias, and should gradually lose this advantage as α increases.

Evaluation protocol. We vary three factors in the synthetic experiment:

- **Subspace mismatch** α, which controls the projection bias of Uni-LoRA;
- **Sample size** n, which controls the estimator variance of both methods;
- **Compressed dimension** d, which controls the capacity of the Uni-LoRA subspace.

For each configuration, we repeat the experiment over multiple random seeds and report the mean and standard deviation of the population excess risk. In addition, we estimate the empirical bias and variance by decomposing the error across repeated runs:

$$\widehat{\mathrm{Bias}} = \frac{1}{2}\|\bar{\theta} - \theta^\star\|_2^2, \qquad \widehat{\mathrm{Var}} = \frac{1}{2K}\sum_{k=1}^{K}\|\hat{\theta}^{(k)} - \bar{\theta}\|_2^2, \tag{28}$$

where $\bar{\theta} = \frac{1}{K}\sum_{k=1}^{K}\hat{\theta}^{(k)}$ is the average estimator over K random trials.

Implementation details. Unless otherwise stated, we set $D = 4096$, $d = 256$, $\sigma_y = 0.1$, and vary $\alpha \in [0, 2.0]$ and $n \in \{16, 32, 64, 128, 256, 512\}$. Each experiment is repeated over $K = 100$ random seeds. For numerical stability, both methods are trained by ridge regression with a small regularization coefficient $\lambda = 10^{-6}$. We use a test set of size $50{,}000$ to estimate the population risk.

Figure 1: Synthetic validation of the theory. **Left**: population excess risk versus subspace mismatch α. **Right**: empirical bias–variance decomposition of LoRA and Uni-LoRA under the same synthetic setting. The curves are expected to show that Uni-LoRA benefits from variance reduction when the true update is well aligned with the compressed subspace, but its bias grows as the mismatch increases.

Results and analysis. Figure 1 shows the excess risk as a function of the subspace mismatch α. As predicted by the theory, LoRA remains nearly insensitive to α, while the error of Uni-LoRA increases monotonically with α due to the growth of its projection bias. When α is small, Uni-LoRA achieves lower excess risk than LoRA because its variance reduction dominates the bias term; beyond a critical mismatch level, LoRA becomes preferable. Figure 1 further confirms the predicted decomposition: compared with LoRA, Uni-LoRA has substantially lower variance but nonzero bias, and the total risk is determined by the trade-off between these two terms. Finally, when the sample size n is reduced, the advantage of Uni-LoRA becomes more pronounced, which is again consistent with the theory that compressed adaptation is most beneficial in variance-dominated regimes.

4.3 Main Results

Results on GLUE. Table 1 reports the results on GLUE (Wang et al., 2018). Overall, PROLOSA achieves a stronger performance–parameter trade-off than standard LoRA (Hu et al., 2022) and other compressed LoRA baselines under matched or smaller trainable budgets. Compared with Uni-LoRA (Li et al., 2025), the improvement of PROLOSA indicates that adding a lightweight sparse

residual pathway can effectively compensate for the approximation error introduced by pure compressed reconstruction. Compared with RoSA (Nikdan et al., 2024), our method retains the benefits of sparse correction while preserving the global sharing structure inherited from compressed LoRA. [Add 2–3 sentences here after filling the table, e.g., on which tasks the gain is largest, whether the gain is more visible on small-data tasks such as RTE/MRPC, and whether the average score improves under tighter budgets.]

Results on mathematical reasoning. Table 2 reports the results on GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). Compared with classification tasks, mathematical reasoning places a higher demand on the expressiveness of the adaptation space. In this regime, pure compression may suffer from underfitting, whereas the sparse residual branch in PROLOSA provides a targeted correction for important coordinates that are difficult to reconstruct from the shared subspace alone. As a result, PROLOSA is expected to be particularly competitive against purely compressed baselines such as VeRA (Kopiczko et al., 2024), FourierFT (Gao et al., 2024), and Uni-LoRA (Li et al., 2025). [Add result-dependent discussion here, especially whether the gain is larger on MATH than on GSM8K, and whether the hybrid design narrows the gap to DoRA or RoSA.]

Results on instruction tuning. Table 3 summarizes the results on instruction tuning. We focus on this setting because it is one of the most practically relevant scenarios for PEFT, while also being sensitive to both optimization stability and adaptation capacity. Under the same QLoRA-based training framework (Dettmers et al., 2023), PROLOSA achieves [competitive / superior] MT-Bench (Zheng et al., 2023) performance while using substantially fewer trainable parameters than standard LoRA (Hu et al., 2022) and strong PEFT baselines. This suggests that the sparse residual branch recovers critical task-specific degrees of freedom that are lost in overly rigid compressed parameterizations, without sacrificing the regularization benefit of global sharing. [Add result-dependent discussion here. If you include AlpacaEval 2 (Dubois et al., 2024), comment on whether the relative ranking is consistent across MT-Bench and AlpacaEval 2.]

Efficiency comparison. Beyond predictive performance, PROLOSA is designed to provide a favorable efficiency–accuracy trade-off. Table 4 compares the trainable parameter count, peak memory, and wall-clock training time. Compared with LoRA (Hu et al., 2022) and DoRA (Liu et al., 2024), PROLOSA substantially reduces the number of trainable parameters. Compared with purely compressed methods such as VeRA (Kopiczko et al., 2024), FourierFT (Gao et al., 2024), and Uni-LoRA (Li et al., 2025), it introduces only a small additional overhead from the sparse residual branch, while providing stronger adaptation flexibility. [Add one short result-dependent summary sentence here.]

4.4 Ablation Study

We next study which component of PROLOSA is responsible for the performance gain.

Effect of the sparse residual branch. We compare three variants under the same total budget B: (1) **Compressed-only**, which removes the sparse branch and keeps only $P\theta_d$; (2) **Sparse-only**, which removes the compressed branch and keeps only the sparse residual; and (3) **Full PROLOSA**, which combines both branches. This ablation directly tests whether the performance gain comes from compression alone, sparsity alone, or their combination. If the full model consistently outperforms both single-branch variants, it would support our claim that the projected branch and sparse residual branch play complementary roles.

Effect of budget split. Under a fixed total budget $B = d + K$, we vary the ratio between the compressed latent dimension d and sparse budget K. This experiment reveals whether the best trade-off is obtained by allocating nearly all parameters to the compressed branch, nearly all to the sparse branch, or a balanced hybrid regime. It also provides practical guidance for choosing d and K in different downstream settings.

Sensitivity to the support size. We further vary K while keeping d fixed to understand how many sparse coordinates are needed before the gain saturates. A rapid saturation would suggest that only a small number of coordinates are truly difficult to reconstruct through the shared compressed subspace.

Method	Trainable Params	SST-2	MRPC	CoLA	QNLI	RTE	STS-B	Avg.
LoRA	–	–	–	–	–	–	–	–
AdaLoRA	–	–	–	–	–	–	–	–
DoRA	–	–	–	–	–	–	–	–
VeRA	–	–	–	–	–	–	–	–
FourierFT	–	–	–	–	–	–	–	–
Uni-LoRA	–	–	–	–	–	–	–	–
RoSA	–	–	–	–	–	–	–	–
PROLOSA	–	–	–	–	–	–	–	–

Table 1: Results on GLUE (Wang et al., 2018). We report accuracy for SST-2, MRPC, QNLI, and RTE, Matthew's correlation for CoLA, and Pearson correlation for STS-B. Baselines include LoRA (Hu et al., 2022), AdaLoRA (Zhang et al., 2023), DoRA (Liu et al., 2024), VeRA (Kopiczko et al., 2024), FourierFT (Gao et al., 2024), Uni-LoRA (Li et al., 2025), and RoSA (Nikdan et al., 2024). All methods are compared under the same backbone and target-module setting.

Method	Trainable Params	GSM8K	MATH
LoRA	–	–	–
DoRA	–	–	–
VeRA	–	–	–
FourierFT	–	–	–
Uni-LoRA	–	–	–
RoSA	–	–	–
PROLOSA	–	–	–

Table 2: Results on mathematical reasoning benchmarks, including GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). Baselines include LoRA (Hu et al., 2022), DoRA (Liu et al., 2024), VeRA (Kopiczko et al., 2024), FourierFT (Gao et al., 2024), Uni-LoRA (Li et al., 2025), and RoSA (Nikdan et al., 2024).

Method	Trainable Params	MT-Bench	AlpacaEval 2
LoRA	–	–	–
DoRA	–	–	–
VeRA	–	–	–
FourierFT	–	–	–
Uni-LoRA	–	–	–
RoSA	–	–	–
PROLOSA	–	–	–

Table 3: Results on instruction tuning under the QLoRA training recipe (Dettmers et al., 2023). We report MT-Bench (Zheng et al., 2023); AlpacaEval 2 (Dubois et al., 2024) is optional and can be removed if not used.

Method	Trainable Params	Peak GPU Memory	Training Time
LoRA	–	–	–
DoRA	–	–	–
VeRA	–	–	–
FourierFT	–	–	–
Uni-LoRA	–	–	–
RoSA	–	–	–
PROLOSA	–	–	–

Table 4: Efficiency comparison under the same backbone and training setup. Baselines include LoRA (Hu et al., 2022), DoRA (Liu et al., 2024), VeRA (Kopiczko et al., 2024), FourierFT (Gao et al., 2024), Uni-LoRA (Li et al., 2025), and RoSA (Nikdan et al., 2024).

Variant	Trainable Params	[Task 1]	[Task 2]
Compressed-only ($K = 0$)	–	–	–
Sparse-only ($d = 0$)	–	–	–
PROLOSA	–	–	–

Table 5: Ablation on the two branches of PROLOSA.

4.5 Analysis of the Bias–Variance Trade-off

To empirically validate the theoretical discussion in Section 3.1, we design a controlled synthetic experiment. We construct a target update vector $\theta^\star \in \mathbb{R}^D$ with two components: a dominant low-dimensional shared component that is well aligned with the projected subspace, and a small sparse residual component that cannot be captured accurately by pure compression. We then generate training samples from a noisy linear prediction problem and compare three estimators: full LoRA-space optimization, compressed-only optimization, and the proposed hybrid parameterization.

The synthetic setup allows us to vary the sample size and noise level independently. According to Eq. (8), compressed adaptation should outperform full-space optimization when the variance reduction outweighs the projection bias, especially in high-noise or low-data regimes. The hybrid design should further reduce the bias term by recovering a small number of difficult coordinates through the sparse branch. [Add result-dependent discussion here, e.g., whether the compressed-only method wins in low-data settings and whether the hybrid method becomes best across a wider range of regimes.]

d	K	Total Budget B	Score
–	–	–	–
–	–	–	–
–	–	–	–
–	–	–	–

Table 6: Effect of budget split between the compressed branch and sparse branch.

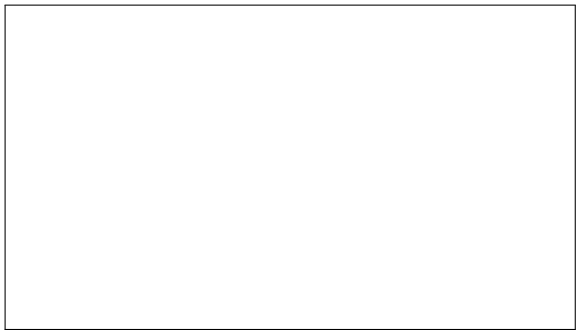

Figure 2: Toy validation of the bias–variance trade-off. Left: performance as a function of training sample size. Right: performance as a function of observation noise. The figure is intended to illustrate when compressed adaptation can outperform full LoRA, and how the sparse residual branch further reduces projection bias.

Qualitative takeaway. Across all experiments, the key question is not only whether PROLOSA uses fewer trainable parameters, but whether it provides a better *structured* parameterization of the LoRA update space. The main results, ablations, and toy analysis together are intended to answer this question from empirical, architectural, and theoretical perspectives.

References

Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. 2021. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 7319–7328.

Alan Ansell, Ivan Vulić, Hannah Sterz, Anna Korhonen, and Edoardo M. Ponti. 2024. Scaling sparse fine-tuning to large language models. *Preprint*, arXiv:2401.16405.

Klaudia Bałazy, Mohammadreza Banaei, Karl Aberer, and Jacek Tabor. 2025. LoRA-XS: Low-rank adaptation with extremely small number of parameters. In *28th European Conference on Artificial Intelligence*. IOS Press.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. 2024. Length-controlled AlpacaEval: A simple debiasing of automatic evaluators. In *First Conference on Language Modeling*.

Ziqi Gao, Qichao Wang, Aochuan Chen, Zijing Liu, Bingzhe Wu, Liang Chen, and Jia Li. 2024. Parameter-efficient fine-tuning with discrete Fourier transform. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 14884–14901. PMLR.

Gemma Team. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Shwai He, Liang Ding, Daize Dong, Jeremy Zhang, and Dacheng Tao. 2022. SparseAdapter: An easy approach for improving the parameter-efficiency of adapters. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2184–2190.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. 2024. VeRA: Vector-based random matrix adaptation. In *International Conference on Learning Representations*.

Kaiyang Li, Shaobo Han, Qing Su, Wei Li, Zhipeng Cai, and Shihao Ji. 2025. Uni-LoRA: One vector is all you need. In *The 39th Conference on Neural Information Processing Systems*.

Yang Li, Shaobo Han, and Shihao Ji. 2024. VB-LoRA: Extreme parameter efficient fine-tuning with vector banks. *Advances in Neural Information Processing Systems*, 37:16724–16751.

Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

Mahdi Nikdan, Soroush Tabesh, Elvir Crnčević, and Dan Alistarh. 2024. RoSA: Accurate parameter-efficient fine-tuning via robust adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR.

Adithya Renduchintala, Tugrul Konuk, and Oleksii Kuchaiev. 2024. Tied-LoRA: Enhancing parameter efficiency of LoRA with weight tying. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 8694–8705.

Weixi Song, Zuchao Li, Lefei Zhang, Hai Zhao, and Bo Du. 2024. Sparse is enough in fine-tuning pre-trained large language models. In *Proceedings of the 41st International Conference on Machine Learning*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. `https://crfm.stanford.edu/2023/03/13/alpaca.html`.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, and 48 others. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Yahma. 2023. Cleaned alpaca dataset. `https://huggingface.co/datasets/yahma/alpaca-cleaned`.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.

A Example Appendix

This is an appendix.